Message Broker & Event-Driven Architecture

Student Name: [Lyan Khaled Bakrman

NPM: 232310045

Class: TI-23-PA1

1. Introduction

This report documents the implementation of asynchronous communication between microservices using **RabbitMQ**. The goal is to apply Event-Driven Architecture (EDA) to decouple the "Book Service" from the logging operations, as required by the Module 4 Learning Tasks.

2. System Architecture & Flow

The system consists of two main services communicating asynchronously:

1. **Publisher (Book Service):** When a user creates/updates/deletes a book via REST API, this service sends a message to RabbitMQ.
2. **Message Broker (RabbitMQ):** Receives the message and places it in the book_created (or similar) queue.
3. **Consumer (Log Service):** Listens to the queue and processes the logs independently.

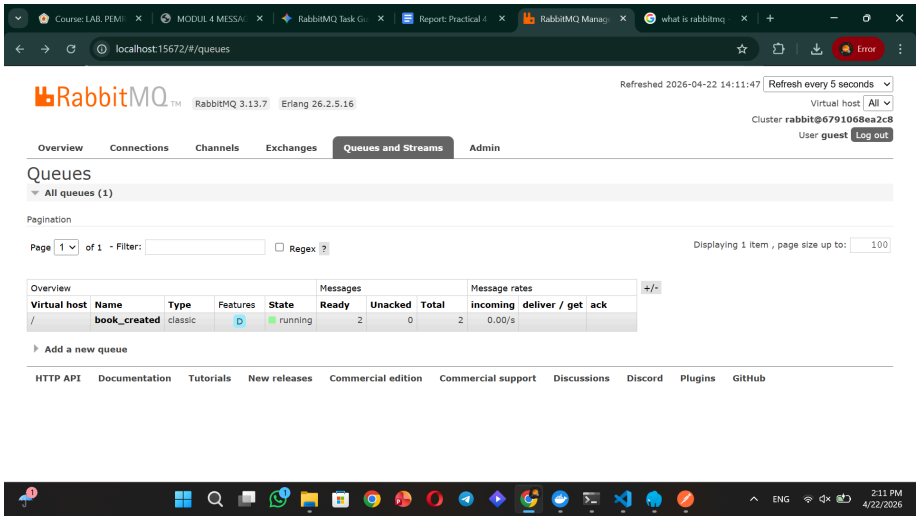
3. Implementation Details

A. Environment Configuration (Docker)

RabbitMQ is deployed using Docker with the management plugin for monitoring.

Command:

```
docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```



B. Code Implementation (Publisher)

Modified the controller logic to include amqplib to send events.

- **Logical Step:** Connect -> Create Channel -> Assert Queue -> Send Buffer.

```
require("dotenv").config();

const amqp = require("amqplib");

class RabbitMQService {

  constructor() {

    this.connection = null;

    this.channel = null;

  }



  async connect() {

    try {

      this.connection = await amqp.connect(

        process.env.RABBITMQ_URL || "amqp://localhost",
```

```
    );

    this.channel = await this.connection.createChannel();

    console.log("Connected to RabbitMQ");

    return true;

  } catch (error) {

    console.error("Failed to connect to RabbitMQ:", error);

    return false;

  }

}



async sendToQueue(queueName, message) {

  try {

    if (!this.channel) {

      await this.connect();

    }



    await this.channel.assertQueue(queueName, {

      durable: true,

    });



    this.channel.sendToQueue(

      queueName,
```

```
        Buffer.from(JSON.stringify(message)),
        {
          persistent: true,
        },
      );

      console.log(`Message sent to queue ${queueName}:`,
message);
      return true;
    } catch (error) {
      console.error("Error sending message to queue:", error);
      return false;
    }
  }

  async close() {
    try {
      if (this.channel) await this.channel.close();
      if (this.connection) await this.connection.close();
      console.log("RabbitMQ connection closed");
    } catch (error) {
      console.error("Error closing RabbitMQ connection:",
```

```
error);

    }

  }

}

module.exports = new RabbitMQService();
```

C. Code Implementation (Consumer)

The Log Service uses a listener to consume messages from the queue.

- **Logical Step:** Connect -> Assert Queue -> Consume message -> Console Log/Save to DB.

```
Connected to RabbitMQ
Message sent to queue book_created: {
  event: 'book.created',
  timestamp: '2026-04-22T07:10:57.077Z',
  data: {
    id: 4,
    title: 'Buku Pemograman Website',
    author: 'Febry D Fairuz',
    is_free: false,
    language: 'id-ID'
  }
}
Executing (default): INSERT INTO `books` (`id`,`title`,`author`,`rating`,`views`,`is_free`,`language`,`sinopsis`,`story`,`image`,`cre
ated_at`,`updated_at`) VALUES (DEFAULT,?,?,?,?,?,?,?,?,?,?,?);
Message sent to queue book_created: {
  event: 'book.created',
  timestamp: '2026-04-22T07:11:22.423Z',
  data: {
    id: 5,
    title: 'Buku Pemograman Website',
    author: 'Febry D Fairuz',
    is_free: false,
    language: 'id-ID'
  }
}
```

4. Verification & Testing

CLI Verification

Using the commands from Section 4.3 of the module to verify the queue status:

Check queue via docker
docker exec -it rabbitmq rabbitmqctl list_queues

Check logs for message arrival
docker logs rabbitmq

```
C:\Users\layan>docker exec -it rabbitmq rabbitmqctl list_queues
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages
book_created    2

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug rabbitmq
    Learn more at https://docs.docker.com/go/debug-cli/
```

Application Execution

Proof that when the Book Service is triggered, the Log Service reacts.

```
}
Executing (default): INSERT INTO `books` (`id`,`title`,`author`,`rating`,`views`,`is_free`,`language`,`sinopsis`,`story`,`image`,`cre
ated_at`,`updated_at`) VALUES (DEFAULT,?,?,?,?,?,?,?,?,?,?,?);
Message sent to queue book_created: {
  event: 'book.created',
  timestamp: '2026-04-22T07:11:22.423Z',
  data: {
    id: 5,
    title: 'Buku Pemograman Website',
    author: 'Febry D Fairuz',
    is_free: false,
    language: 'id-ID'
  }
}
```

```
Available Endpoints:
 - Books API: http://localhost:8080/api/books
 - Users API: http://localhost:8080/api/users
 - Health Check: http://localhost:8080/health
====================================================================

2026-04-22T07:10:56.595Z [EG:log-policy] info: 📬 Book Service Request: POST /api/books/
(node:4612) [DEP0060] DeprecationWarning: The `util._extend` API is deprecated. Please use Object.assign() instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
2026-04-22T07:11:22.382Z [EG:log-policy] info: 📬 Book Service Request: POST /api/books/
2026-04-22T07:22:29.774Z [EG:log-policy] info: 📬 Book Service Request: POST /api/books/
```

5. Conclusion

By implementing RabbitMQ, the services are decoupled. The Book Service remains responsive even if the Log Service is processing data slowly, fulfilling the requirements of a scalable Event-Driven Architecture.